

Embedded Firmware Design

Juan Pablo Villamil Miranda, Kevin Mateo Vargas Moreno, Santiago Andrés García Ortiz y Valentina Yepes Ramírez

Diagrama de bloques del sistema embebido:

El sistema embebido está organizado alrededor del microcontrolador, el cual actúa como unidad central de control. Este recibe señales provenientes de los interruptores, sensores y módulo RTC/logging, además de comunicarse con la interfaz de usuario mediante USB. El microcontrolador controla tres etapas principales de movimiento, correspondientes a los ejes X, Y y Z. Cada eje cuenta con una etapa de aislamiento, una etapa de potencia y un motor paso a paso. La etapa de aislamiento permite separar eléctricamente las señales de control del microcontrolador respecto a la etapa de potencia, mientras que la etapa de potencia entrega la corriente necesaria para accionar cada motor.

Adicionalmente, el sistema cuenta con un bloque independiente para el control de la herramienta de corte. Este bloque incluye una etapa de aislamiento, un circuito de encendido y apagado, y la herramienta de corte, alimentada mediante una fuente externa para el circuito de potencia del mecanizado. El sistema también utiliza fuentes de alimentación separadas para el control, el movimiento y la herramienta de corte. Esta separación permite reducir interferencias eléctricas y mejorar la seguridad del sistema, especialmente al trabajar con cargas de potencia.

En conjunto, el diagrama muestra la relación entre los módulos de control, sensado, potencia, comunicación y actuación, permitiendo visualizar la arquitectura general del sistema antes de detallar el firmware y las conexiones electrónicas.

Descripción por bloques:

- Microcontrolador: coordina la lectura de sensores, control de motores, manejo de errores, comunicación con la interfaz y registro de eventos.
- Interruptores y sensores: permiten detectar condiciones de operación, finales de carrera, eventos de seguridad o variables relevantes del sistema.
- RTC y logging: permite registrar eventos del sistema con referencia temporal.
- USB/interfaz: permite la comunicación entre el sistema embebido y la interfaz gráfica.
- Circuitos de aislamiento: protegen al microcontrolador y separan las señales de control de las etapas de potencia.
- Circuitos de potencia: entregan la energía necesaria para accionar motores y herramientas.
- Motores X, Y, Z: ejecutan el movimiento de los tres grados de libertad de la CNC.
- Herramienta de corte: actuador encargado del proceso de mecanizado.
- Fuentes de alimentación: suministran energía de forma separada al control, movimiento y mecanizado.

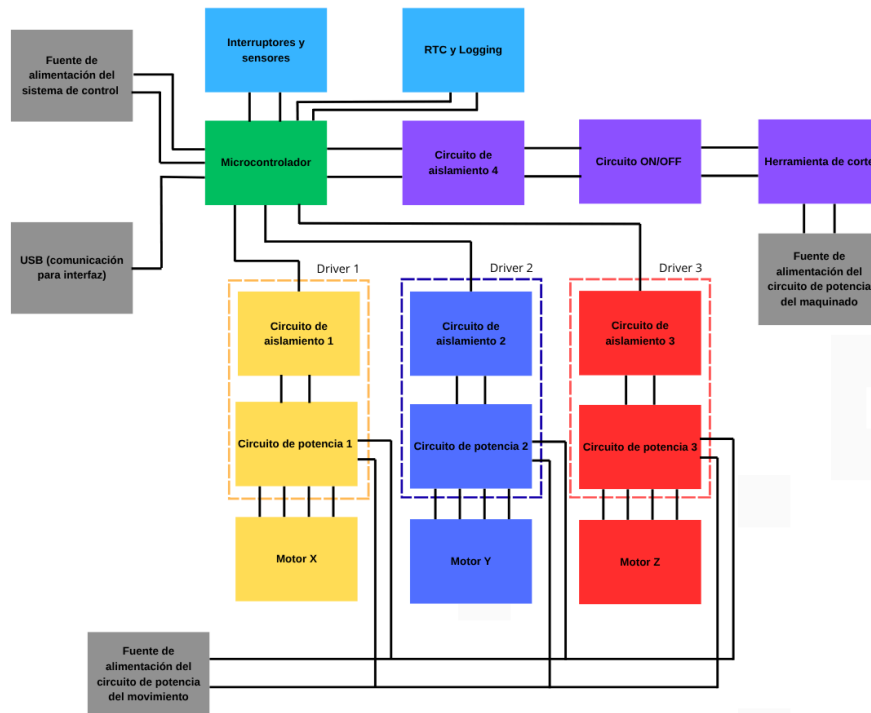


Figura 1. Diagrama de bloques.

Arquitectura de firmware:

La programación modular es una herramienta que permite dividir un problema en varios subproblemas, o denominados módulos, para ser resueltos independientemente. Una vez desarrollados, estos se unen nuevamente para solucionar el problema inicial. En términos del lenguaje de programación C, se crearon varios archivos .c, en los cuales se encuentran funciones encargadas de realizar tareas específicas, como inicializar los pines GPIO de la ESP32, activar o detener todos los motores, leer un sensor, entre otras. Asimismo, cada archivo .c cuenta con su respectivo archivo de cabecera (.h), en el que se declaran las funciones con sus argumentos, además de macros o máscaras para el compilador, las librerías necesarias para trabajar con la ESP32 (temporizadores, PWM, etc.) y tipos de datos en C.

Para integrar posteriormente estos archivos en el programa principal (main.c), la estructura para cómo resolver este problema en su totalidad está dada por una máquina de estados, donde la función a usar depende del estado actual en el que se encuentre el programa. La transición de un estado al otro está ligada tanto a los botones de una interfaz (GUI), creada en lenguaje de programación Python, como a condiciones de emergencia y fin de la tarea solicitada.

En cuanto a lo que brinda la elección de esta arquitectura, por un lado, con la programación modular, se obtiene una mejor organización y navegación del programa, puesto que es más sencillo tratar cada elemento cuando se tiene un código por separado, específico para una sola tarea. De esta manera, se facilita a su vez el mantenimiento y la documentación del código, así como también su reutilización; mientras que, por otro lado, la máquina de estados dota al microcontrolador de una metodología a seguir y una definición clara de las

acciones a realizar para cada momento en el programa, de forma que se descartan todos los casos posibles diferentes a los estipulados en el diagrama de estados, como se observa en la tabla 1.

Estado Actual	Entrada / Evento (GUI)	Estado Siguiente	Acción del Sistema Firmware
INIT	Al encender la ESP32	IDLE	Se llaman las funciones de inicialización de los GPIO, PWM, temporizadores y declaraciones de variables. Una vez terminado, se deja en estado de reposo (IDLE) o en espera de una nueva orden el sistema.
IDLE	+X, -X, +Y, -Y, +Z o -Z en GUI	JOG	Una vez se haya determinado cuánto se moverá el motor [mm/min] en GUI, dependiendo del eje seleccionado, se activa el respectivo motor mientras se presiona su botón. Al soltarlo, el sistema regresa al estado de reposo. Es importante considerar que al iniciar la máquina, se deben desplazar un poco los tres motores de los ejes para que dejen de tener contacto constante con los inicios de carrera. Si no, no podrá realizar la función solicitada en RUNNING.
IDLE	Start en GUI	RUNNING	Los motores de los ejes X y Y comienzan a moverse

			conjuntamente para generar la forma de un circuito sobre la pieza que se esté trabajando.
RUNNING	Botón Pause en GUI	PAUSE	Motores de los ejes detenidos pero el del spindle sigue encendido.
PAUSE	Botón Start en GUI	RUNNING	Se continúa el proceso justo donde la máquina había quedado.
RUNNING	Condiciones de límite de corriente en motores, inicios y finales de carrera o botón Stop en GUI	ALARM	Si se superan los rangos para una operación segura o se presiona el botón en la interfaz, se detienen todos los motores y, tras 2 segundos, el spindle.
ALARM	Botón Reset en GUI	HOMING	Se devuelve primero el motor del eje Z hasta tocar el inicio de carrera. Al llegar ahí, se activan X y Y para regresar también hasta sus inicios de carrera.
HOMING	Inicios de carrera de los motores de los ejes	IDLE	Al encontrarse los motores en sus posiciones iniciales de la máquina, se deja en estado de reposo el sistema.

--	--	--	--

Tabla 1: Diagrama de estados para el firmware.

Estrategia de manejo de errores: (Pablo)

En la implementación física de la etapa electrónica, se buscó minimizar los errores asociados a las conexiones mediante un proceso sistemático de verificación. Inicialmente, se comprobaba en la PCB que existiera continuidad únicamente en las rutas correspondientes y que no hubiera conexiones no deseadas entre pistas.

Una vez validada esta etapa, se procedía a realizar las conexiones sin incluir aún los componentes electrónicos más sensibles, como la ESP32 o el amplificador operacional. Esto permitía medir y verificar que los valores de voltaje obtenidos fueran los adecuados y coincidieran con los esperados para el correcto funcionamiento del sistema.

En caso de detectar valores incorrectos o comportamientos no esperados, se realizaba nuevamente una revisión detallada de las conexiones y de las rutas de la PCB hasta identificar y corregir la falla.

Adicionalmente, se procuró realizar todas las conexiones de manera firme y segura, garantizando un buen contacto eléctrico entre los conductores. Para ello, se verificó la continuidad de las conexiones mediante el uso de un multímetro, asegurando así la correcta transmisión de las señales y la alimentación del sistema.

En cuanto a los errores que pudieran presentarse una vez realizado el montaje del sistema, se implementaron mediante firmware dos mecanismos principales de seguridad.

El primero corresponde a los finales de carrera, los cuales garantizan que, en caso de que el programa exceda el espacio de trabajo físico del sistema o se presente cualquier otra falla relacionada con el desplazamiento, estos sean accionados y provoquen la detención inmediata de la ejecución del programa.

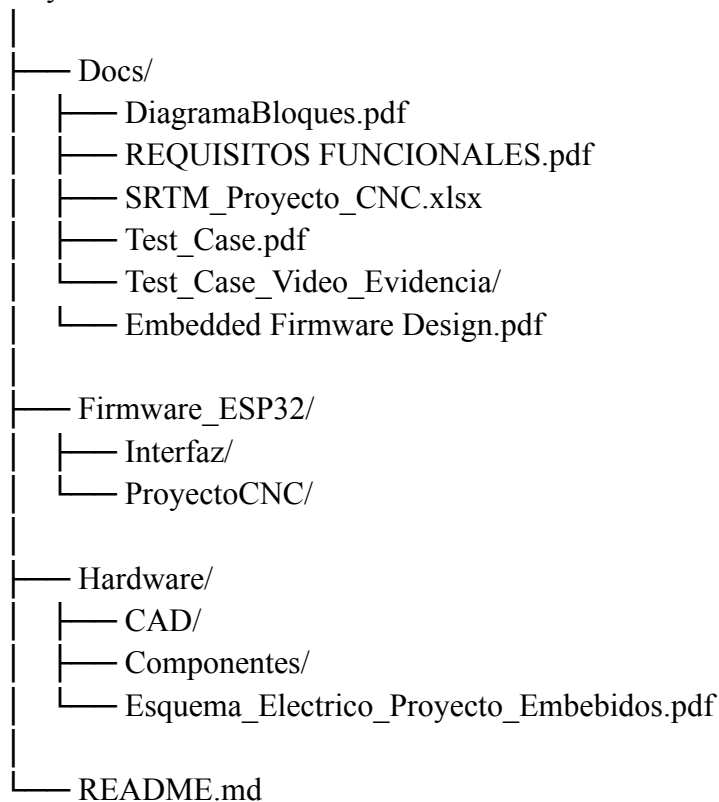
Como segundo mecanismo de seguridad, se incorporan sensores de corriente en cada uno de los motores. Estos permiten supervisar el consumo de corriente de manera constante, garantizando que, en caso de presentarse una sobrecorriente que pudiera afectar la vida útil de los motores o comprometer el funcionamiento del sistema, el programa detenga su ejecución de forma preventiva.

Estructura de directorios del proyecto y flujo de trabajo del repositorio:

El repositorio del proyecto fue organizado en diferentes carpetas según el tipo de información y componentes desarrollados, con el objetivo de mantener una estructura clara y facilitar la trazabilidad entre documentación, firmware, hardware y evidencias del sistema.

La estructura principal del repositorio es la siguiente:

Proyecto Embebidos/



Descripción de directorios:

- Docs: contiene la documentación técnica del proyecto, incluyendo requisitos funcionales, diagramas de bloques, matriz SRTM, casos de prueba y evidencias de validación.
- Firmware_ESP32: contiene el firmware desarrollado para el microcontrolador ESP32 y los archivos relacionados con la interfaz del sistema.
- Hardware: contiene los archivos asociados al diseño físico y electrónico del sistema, incluyendo modelos CAD, documentación de componentes y esquemáticos eléctricos.
- README.md: contiene la descripción general del proyecto, introducción, alcance, objetivos y organización del equipo.

Proyecto Embebidos/El flujo de trabajo del repositorio se realizó mediante GitHub. Cada integrante subió los archivos correspondientes a su rol, mientras que el Technical Lead revisó y aprobó los cambios. Esto permitió mantener una organización centralizada del proyecto y asegurar que los documentos, el firmware, el hardware y las evidencias estuvieran relacionados con los requisitos definidos.

El flujo de trabajo del repositorio se realizó mediante GitHub, donde cada integrante desarrolló y subió archivos relacionados con su rol dentro del proyecto. Posteriormente, los

cambios eran revisados técnicamente antes de ser integrados a la rama principal del repositorio. En caso de encontrarse inconsistencias o errores, se realizaban las correcciones necesarias antes de aprobar la integración final. Esta metodología permitió mantener una organización adecuada del proyecto y asegurar coherencia entre la documentación, el firmware, el hardware y las pruebas desarrolladas.

Estrategia de logging:

Gracias a la implementación del componente DS1307, un Reloj de Tiempo real (RTC), es posible obtener e indicar la hora actual (timestamp) para mostrar junto con los mensajes del tipo Logging, es decir, información (INFO), advertencia (WARN) y error (ERROR). En base a esto, el usuario podrá conocer el estado en el que se encuentra el proceso de su máquina, advertencias en cuanto a pérdida de pasos, si hay error en la conexión del mismo RTC, sobrecorrientes en los motores, entre otros.

Para lograr esto, se cuenta con una función variádica con el fin de ingresar la cantidad de argumentos necesarios, que se deban convertir a un mensaje para enviar y mostrar en la GUI. Dentro de esta función, se genera una etiqueta inicial entre corchetes (para ser reconocida en la GUI de Python), que hace referencia al tipo de logging. A partir de aquí, mediante la librería <stdarg.h>, se crea una lista de argumentos variables que llena un buffer para completarlo con caracteres de la frase con el mensaje deseado. Al final, se añade un salto de línea y carácter nulo para dar a entender a la interfaz que el mensaje está completo y listo para mostrar.

Gestión de comunicaciones: (Pablo)

En cuanto a la gestión de comunicaciones del sistema, se implementaron distintos protocolos de comunicación según las necesidades de cada dispositivo y módulo empleado.

Para la comunicación entre la interfaz gráfica desarrollada en Python y el sistema embebido programado en lenguaje C sobre la ESP32, se utilizó el protocolo UART. Este permitió establecer una comunicación serial confiable para el envío y recepción de datos entre ambos entornos de programación, facilitando la transmisión de comandos, estados y variables necesarias para la operación del sistema.

Por otra parte, la comunicación con el módulo de reloj en tiempo real (RTC) se realizó mediante el protocolo I2C. A través de este protocolo, la ESP32 puede leer y actualizar la información de fecha y hora suministrada por el RTC DS1307, utilizando las funciones implementadas en el archivo `rtc_ds1307.c`.